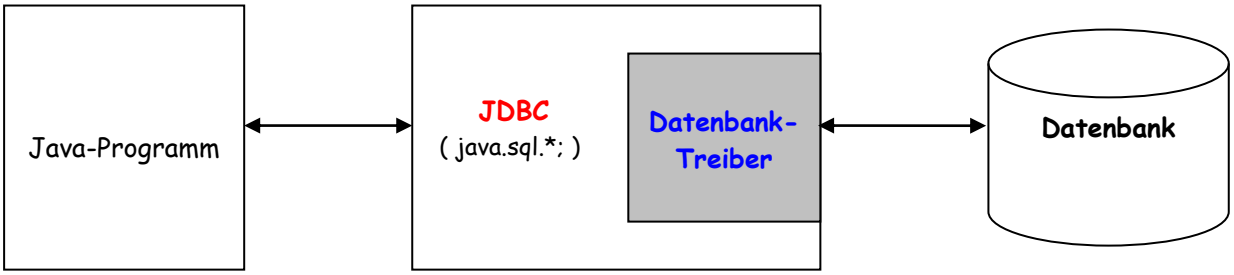
JDBC

Java Database Connectivity (JDBC) ist eine Datenbankschnittstelle der Java-Plattform, die eine einheitliche Schnittstelle zu Datenbanken verschiedener Hersteller bietet und speziell auf relationale Datenbanken ausgerichtet ist.

JDBC ist in seiner Funktion als universelle Datenbankschnittstelle vergleichbar mit z. B. ODBC unter Windows oder DBI unter Perl.

Zu den Aufgaben von JDBC gehört es, Datenbankverbindungen aufzubauen und zu verwalten, SQL-Anfragen an die Datenbank weiterzuleiten und die Ergebnisse in eine für Java nutzbare Form umzuwandeln und dem Programm zur Verfügung zu stellen.



Ablauf und wichtige Klassen der JDBC-API

Vor der Benutzung muss in einem ersten Schritt die entsprechende „Treiber-Klasse" geladen werden. Dies erfolgt durch den Aufruf:

```
Class.forName ( "Treibername")
```

Datenbank	Treibername
MySQL	com.mysql.jdbc.Driver
Access (über JDBC-ODBC-Bridge)	sun.jdbc.odbc.JdbcOdbcDriver
Oracle	oracle.jdbc.driver.OracleDriver

Liegt der Treiber als extra JavaArchiv (Jar) vor muss dieser noch importiert bzw. im CLASSPATH eingetragen werden. (z.B. bei mysql und eclipse: Beim Projekt unter Java-Erstellunspfad/Libraries das Archiv **mysql-connector-java-5.1.34-bin.jar** hinzufügen)
Jede geladene Treiber-Klasse registriert sich selbstständig bei einem DriverManager der JBDC-API (Paket java.sql.*;)
Über den **DriverManager** kann man Verbindungen (Connections) zu der entsprechenden Datenbank aufbauen.
Die Methode GetConnection(…) liefert dabei ein Objekt des Datenbanktreibers zurück, das eine Implementierung des von der der JDBC-API vorgegebenen Interfaces Connection repräsentiert.

DriverManager
+ getConnection(url:String, user:String, password:String):Connection + registerDriver(driver:Driver) + deregisterDriver(driver:Driver) + …

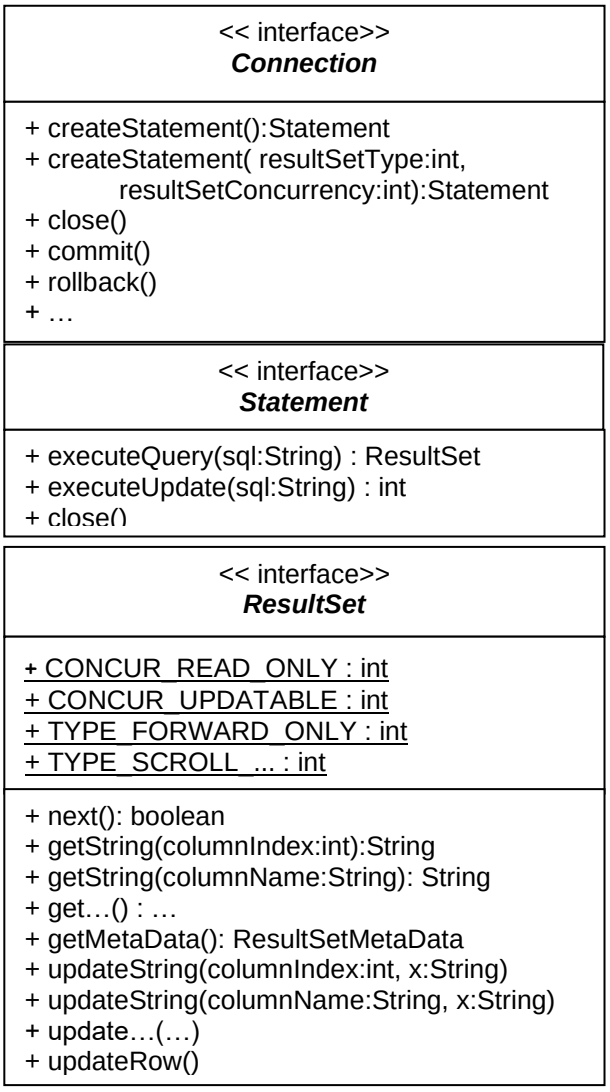
Bei der Abfrage des Connection-Objekts muss über eine spezielle URL die Datenquelle angegeben werden: `jdbc:Subprotokoll:Datenquellenname`
Beispiele:

Datenbank	Subprotokoll	Beispiel
MySQL	mysql	jdbc:mysql://host/database
ODBC-Datenquellen	odbc	jdbc:obc:NameODBCDatenquelle

Über dieses **Connection**-Objekt kann dann ein Statement-Objekt erzeugt werden, über das das Auslesen/Ändern von Werten aus/in der Datenbank ermöglicht wird.
Durch die Angabe von Zusatzparameter (Konstanten aus dem Interface ResultSet) kann das Verhalten gesteuert werden (z.B. nur Read_Only)

Über **Statement**-Objekte können dann SQL-Anfragen an die Datenbank abgesetzt werden. Als Rückgabe erhält man ein ResultSet-Objekt, das die zurück gelieferten Daten aus der Datenbank enthält.

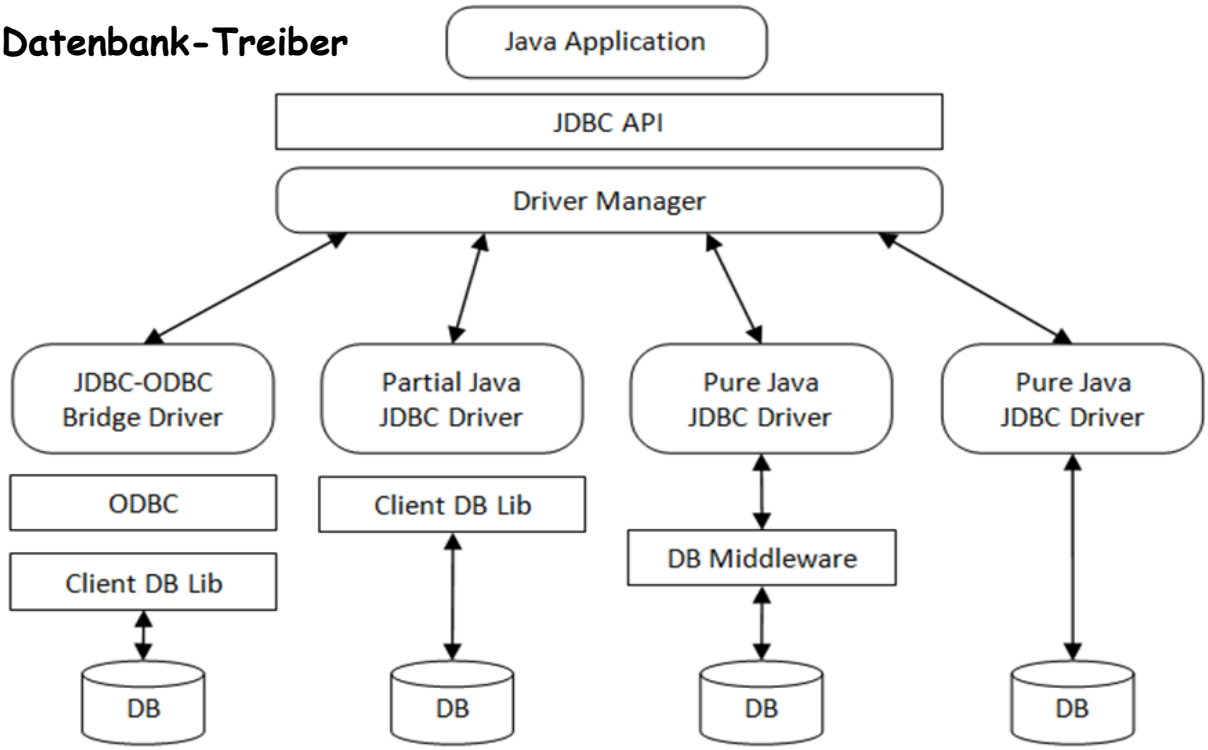
Das **ResultSet**-Objekt kann dann u.a. zeilenweise (Umschaltung auf die nächste Zeile mit next()) ausgelesen werden, wobei alle Spalten der Rückgabetabelle über einen Index oder Spaltennamen adressiert werden können. Es stehen für Standarddatentypen entsprechende Abfragemethoden zur Verfügung.
Die Tabellenstruktur kann über getMetaData() abgefragt/ausgewertet werden.
Auch Änderungen von Werten in der Datenbank sind über das ResultSet-Objekt möglich.



Beispiel: Java Quelltext zur Abfrage von Daten aus MySQL-Datenbank (XAMPP-Beispieltabelle, bei einer eigenen DB ist bei getConnection/executeQuery entsprechend andere Namen anzugeben)

```
try{
  // Treiberklasse laden
  Class.forName( "com.mysql.jdbc.Driver" );

  // Verbindung zur lokalen MySQL-Datenbank herstellen
  Connection con = DriverManager.getConnection("jdbc:mysql://localhost/cdcol",
                                                "root", "");

  // Statement erzeugen
  Statement stmt = con.createStatement();

  // SQL-Abfrage an DBMS
  ResultSet rs = stmt.executeQuery("SELECT * FROM cds");

  // Auslesen der Spalte 1 aus allen Zeilen der Rückgabetabelle im ResultSet
  while( rs.next() ){
      System.out.println( rs.getString(1) );
  }
}
catch(ClassNotFoundException e){
      System.out.println("Fehler beim laden MySQL-Treiber:"+ e.getMessage());
}
catch(SQLException e){
      System.out.println("SQLException: " + e.getMessage());
      System.out.println("SQLState: " + e.getSQLState());
      System.out.println("VendorError: " + e.getErrorCode());
}
```

Datenbank-Treiber



Für jede spezifische Datenbank sind eigene Treiber erforderlich, die die JDBC-Spezifikation implementieren. In der JDBC-Spezifikation werden verschiedene Typen von JDBC-Treibern unterschieden.

- **Typ-1-Treiber:** Ein JDBC-Typ-1-Treiber kommuniziert ausschließlich über einen JDBC-ODBC-Bridge-Treiber. Die bekannteste JDBC-ODBC-Bridge ist die von Sun vertriebene. Damit ist ein Typ-1-Treiber abhängig von einem installierten ODBC-Treiber. Der JDBC-ODBC-Bridge-Treiber wandelt JDBC- in ODBC-Anfragen um.
  Ein Typ-1-Treiber wird dann verwendet, wenn es zu der Datenbank keine eigenständigen JDBC-Treiber gibt.
- **Typ-2-Treiber:** Ein Typ-2-Treiber kommuniziert über eine plattformspezifische Programmbibliothek auf dem Client mit dem Datenbankserver. Das bedeutet, dass für jede Betriebssystem-Plattform zu dem Typ-2-Treiber eine zusätzliche Programmbibliothek benötigt wird.
- **Typ-3-Treiber:** Mittels des Typ-3-Treibers werden die JDBC-API-Befehle in generische DBMS-Befehle übersetzt und (über ein Netzwerkprotokoll) an einen Middleware-Treiber auf einem Anwendungsserver übertragen. Erst dieser Anwendungsserver transformiert die Befehle für die spezifischen Datenbankserver und leitet sie an diese weiter. Ein Typ-3-Treiber benötigt damit keine plattformspezifischen Bibliotheken und muss auch nichts über den verwendeten Datenbankserver wissen.
  Typ-3-Treiber eignen sich sehr gut für Internet-Protokolle im Zusammenhang mit Firewalls.
- **Typ-4-Treiber:** Beim Typ-4-Treiber werden die JDBC-API-Befehle direkt in DBMS-Befehle des jeweiligen Datenbankservers übersetzt und (über ein Netzwerkprotokoll) an diesen übertragen. Ein Middleware-Treiber wird dabei nicht verwendet. Damit kann ein Typ-4-Treiber schneller als ein Typ-3-Treiber sein, ist aber weniger flexibel.
  Typ-4-Treiber eignen sich gut für Intranet-Lösungen, die schnelle Netzprotokolle nutzen wollen.

Beispiele für Schreiboperationen

Update von Datenbank-Zeilen

```
try {
    Statement stmt = connection.createStatement();

    // Prepare a statement to update a record
    String sql = "UPDATE my_table SET col_string='a new string' WHERE
                  col_string = 'a string'";

    // Execute the Update statement
    int updateCount = stmt.executeUpdate(sql);
    // updateCount contains the number of updated rows
} catch (SQLException e) {
}
```

Einfügen einer Zeile in eine Datenbank

```
try {
    Statement stmt = connection.createStatement();

    // Prepare a statement to insert a record
    String sql = "INSERT INTO my_table (col_string) VALUES('a string')";

    // Execute the insert statement
    stmt.executeUpdate(sql);
} catch (SQLException e) {
}
```

Erzeugen änderbarer ResultSets

Änderbare ResultSets erlauben Änderungen der Datenbanktabelle direkt über Operationen des ReslutSets. Um änderbare ResultSets zu erzeugen muss bei Erzeugung des Statement-Objekts der „Concurrency-Typ" `ResultSet.CONCUR_UPDATABLE` angegeben werden.

Update einer Datenbankzeile mit änderbarem ResultSet

Zuerst neuen Wert über `ResultSet.updateXXX()` in das ResultSet-Objekt eintragen, dann `ResultSet.updateRow()` aufrufen um die Änderungen in die Datenbank zu übertragen.

```
try {
    // Create an updatable result set
    Statement stmt = connection.createStatement(
        ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_UPDATABLE);
    ResultSet resultSet = stmt.executeQuery("SELECT * FROM my_table");

    // Move cursor to the row to update
    resultSet.first();

    // Update the value of column col_string on that row
    resultSet.updateString("col_string", "new data");

    // Update the row; if auto-commit is enabled, update is committed
    resultSet.updateRow();
} catch (SQLException e) {
}
```

Aber Achtung: Nicht alle Datenbanken unterstützen dieses Vorgehen!

```
    DatabaseMetaData dmd = connection.getMetaData();
    if (dmd.supportsResultSetConcurrency(
        ResultSet.TYPE_FORWARD_ONLY, ResultSet.CONCUR_UPDATABLE)) {
        // Updatable result sets are supported
    } else {
        // Updatable result sets are not supported
    }
```